



8.1. Data Visualization: Categorical Plots

Previous Section:

 [8. Data Visualization](#)

Contents:

[1. Bar Chart](#)

[2. Clustered Bar Chart](#)

[3. Stacked Bar Chart](#)

[4. Point Plot](#)

[5. Line Chart](#)

[6. Area Chart](#)

[7. Pie Chart](#)

[Summary](#)

[References](#)

Categorical plots are designed to summarize and compare values across different categories. They are among the most commonly used visualizations in exploratory data analysis because they allow us to quickly compare groups, identify trends, and communicate results clearly.

Throughout this section, we will use a **synthetic student dataset** consisting of **200 samples** with the following attributes: ***Student ID; Age; Height; Weight; Gender; Major; GPA***

```

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)

n = 200

df = pd.DataFrame({
    "Student ID": [f"S{i:03d}" for i in range(1, n+1)],
    "Age": np.random.randint(18, 24, n),
    "Height": np.random.normal(170, 8, n).round(1),
    "Weight": np.random.normal(65, 12, n).round(1),
    "Gender": np.random.choice(["Male", "Female"], n),
    "Major": np.random.choice(["AI", "IT", "CSC", "SE"],
n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

df.head()

```

1. Bar Chart

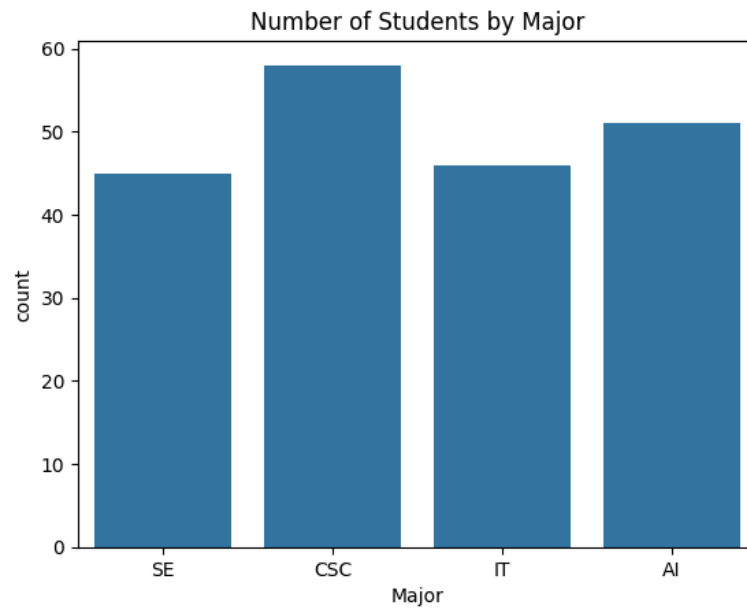
A **bar chart** displays aggregated values for different categories using rectangular bars. The height (or length) of each bar represents the magnitude of the corresponding category, making it one of the simplest and most effective charts for comparing categorical data.

Bar charts are commonly displayed in two orientations:

- **Vertical Bar Chart** – Categories are placed along the x-axis and values are represented by the height of each bar.
- **Horizontal Bar Chart** – Categories are placed along the y-axis and values are represented by the length of each bar.

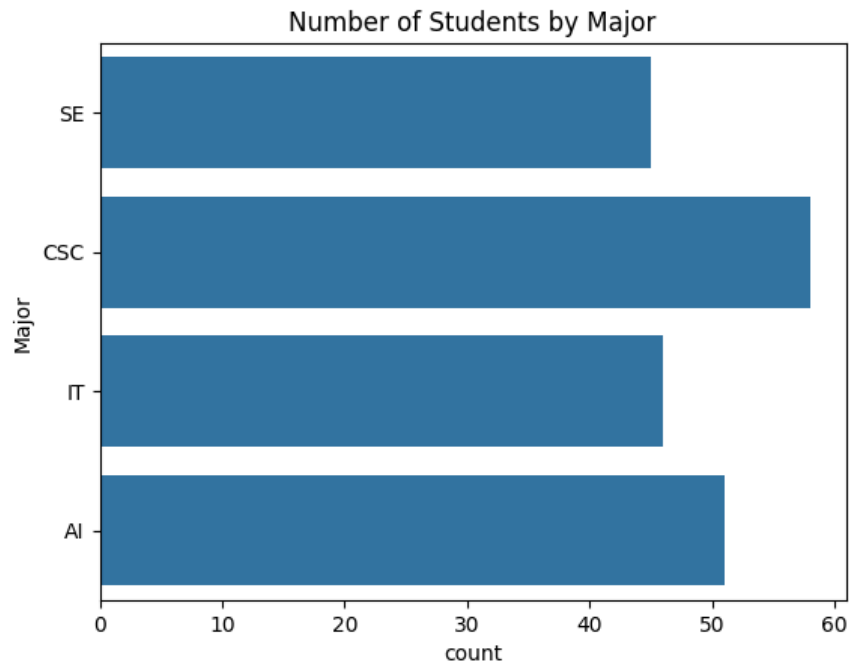
Example 1. Student Count by Major

```
sns.countplot(data=df, x="Major")  
plt.title("Number of Students by Major")  
plt.show()
```



Example 2. Horizontal Bar Chart

```
sns.countplot(data=df, y="Major")  
plt.title("Number of Students by Major")  
plt.show()
```

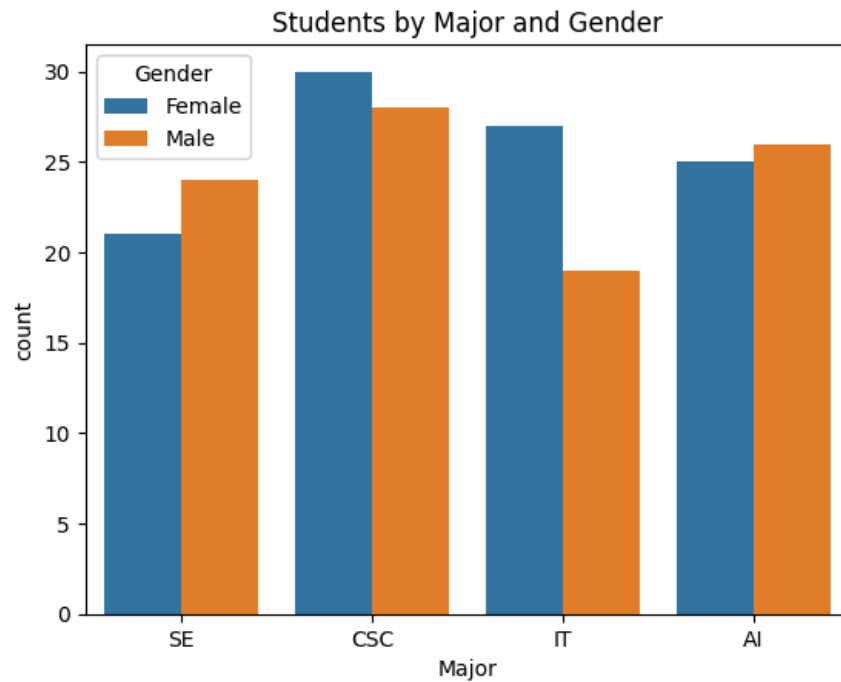


2. Clustered Bar Chart

A **clustered bar chart** (also called a grouped bar chart) compares multiple groups within each category by placing bars side-by-side. This makes it easy to compare subgroups while still preserving the overall category structure.

For example, we can compare the number of **male and female students** within each major.

```
sns.countplot(data=df, x="Major", hue="Gender")  
plt.title("Students by Major and Gender")  
plt.show()
```



3. Stacked Bar Chart

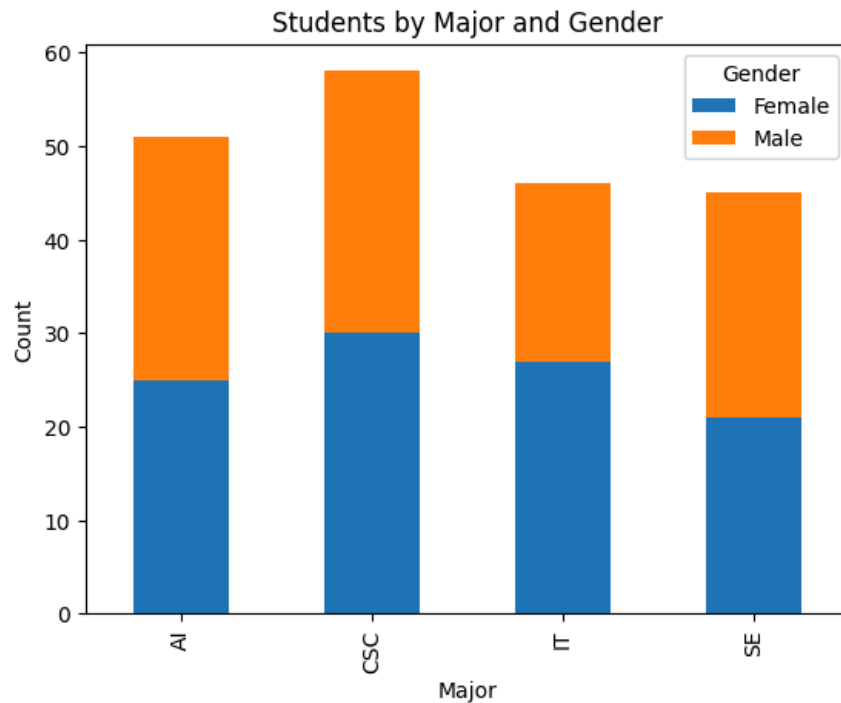
A **stacked bar chart** displays both the total value of each category and the contribution of its individual components. Instead of placing bars side-by-side, each subgroup is stacked vertically, making it easier to visualize proportions within the total.

Since Seaborn does not provide a stacked bar chart directly, we first aggregate the data using Pandas and then plot it.

```
major_gender = pd.crosstab(df["Major"], df["Gender"])

major_gender.plot(kind="bar", stacked=True)

plt.title("Students by Major and Gender")
plt.ylabel("Count")
plt.show()
```



4. Point Plot

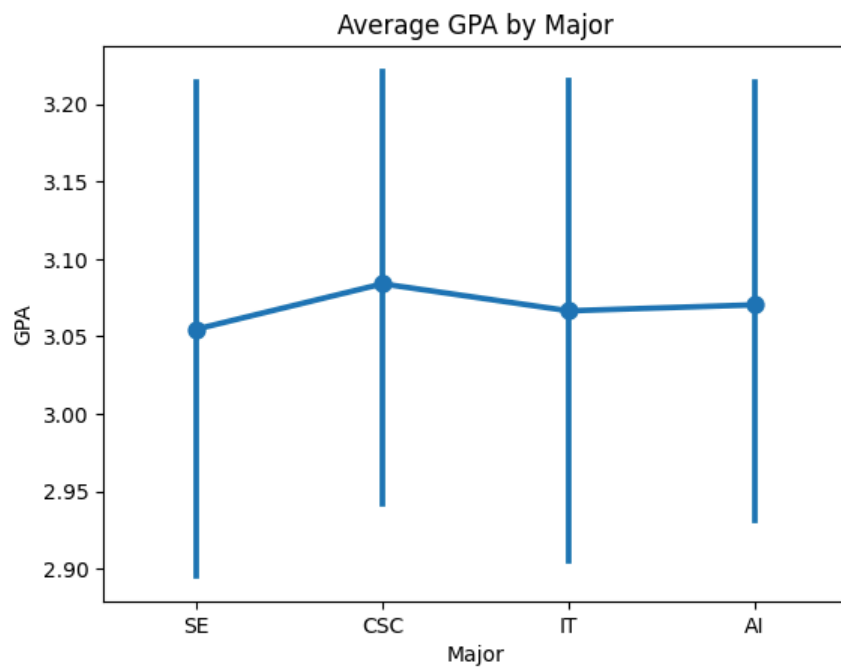
A **point plot** summarizes a numerical variable by displaying its average value for each category. Each point represents the mean of the observations in a category, while connecting lines help visualize differences or trends between categories.

This plot is particularly useful when comparing average values instead of raw counts.

Example 1. Average GPA by Major

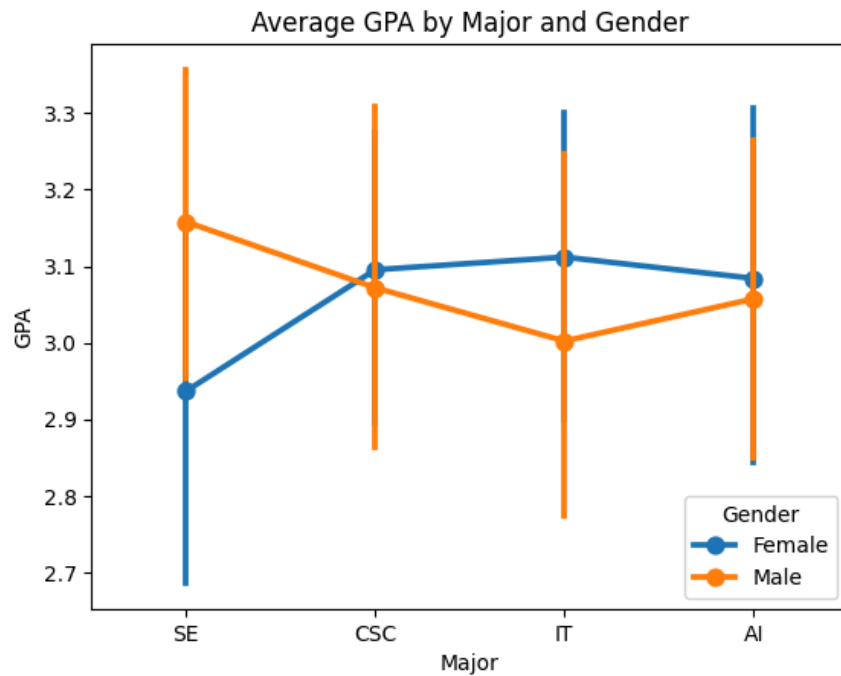
```
sns.pointplot(data=df, x="Major", y="GPA")

plt.title("Average GPA by Major")
plt.show()
```



Example 2. Average GPA by Major and Gender

```
sns.pointplot(  
    data=df,  
    x="Major",  
    y="GPA",  
    hue="Gender"  
)  
  
plt.title("Average GPA by Major and Gender")  
plt.show()
```



5. Line Chart

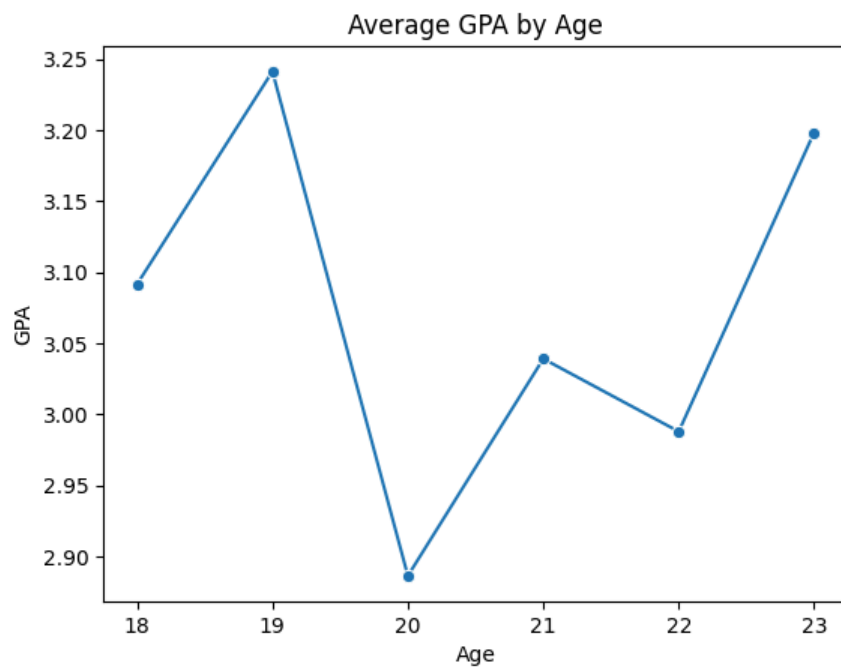
A **line chart** displays data points connected by straight lines, making it ideal for visualizing trends over time or another continuous sequence. While line charts are most commonly used for time-series data, they can also illustrate changes across ordered categories.

Suppose we want to observe how the **average GPA changes with age**.

```
avg_gpa = (
    df.groupby("Age")["GPA"]
      .mean()
      .reset_index()
)

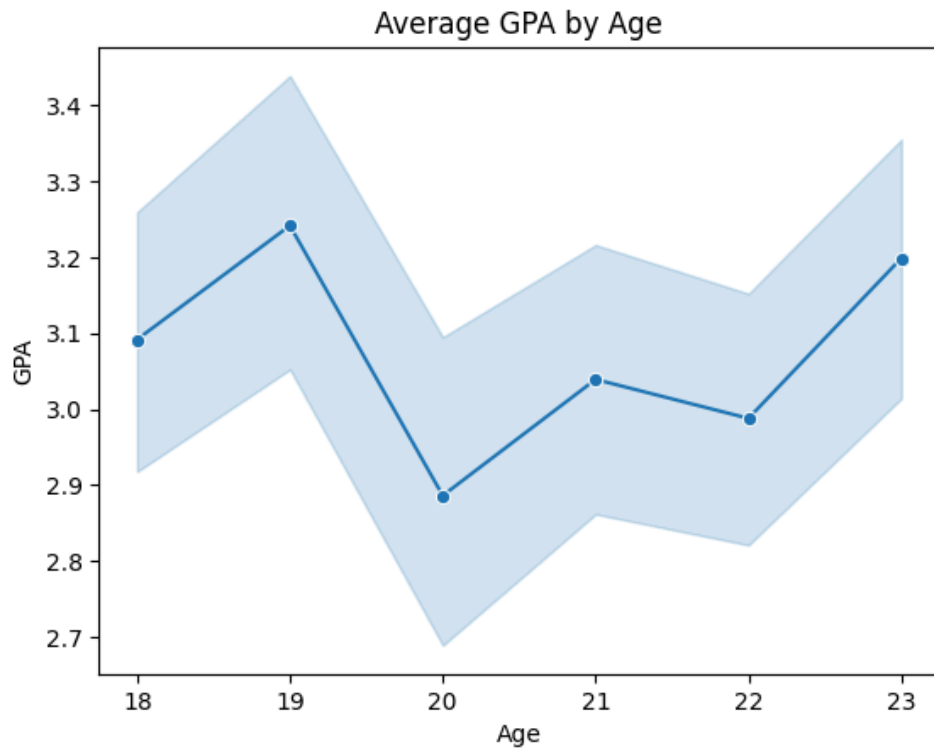
sns.lineplot(data=avg_gpa, x="Age", y="GPA", marker="o")

plt.title("Average GPA by Age")
plt.show()
```

Alternatively, Seaborn can perform the aggregation automatically:

```
sns.lineplot(  
    data=df,  
    x="Age",  
    y="GPA",  
    estimator="mean",  
    marker="o"  
)  
  
plt.title("Average GPA by Age")  
plt.show()
```

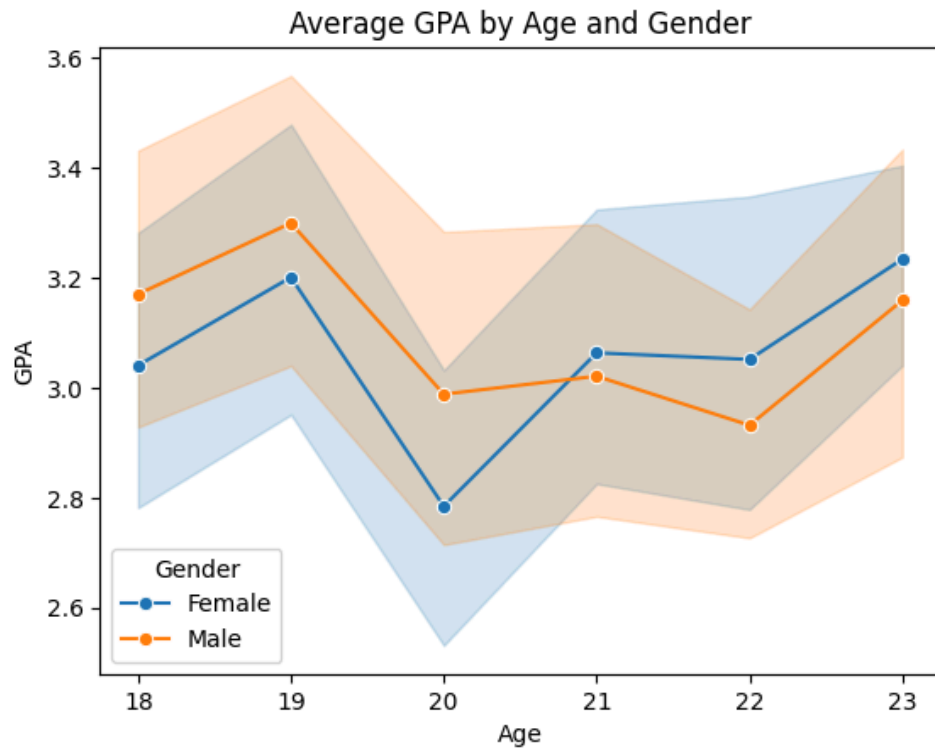


This approach is concise and is generally preferred when the data only requires simple aggregation before visualization.

Another Example. Average GPA by Age and Gender

The `hue` parameter separates the data into multiple groups, allowing us to compare trends between categories on the same figure.

```
sns.lineplot(  
    data=df,  
    x="Age",  
    y="GPA",  
    hue="Gender",  
    estimator="mean",  
    marker="o"  
)  
  
plt.title("Average GPA by Age and Gender")  
plt.show()
```



6. Area Chart

An **area chart** is similar to a line chart but fills the area beneath the line, making it easier to visualize cumulative values and overall trends. Area charts are particularly useful for showing how quantities change over time while emphasizing the magnitude of the values.

When multiple data series are displayed together, they are typically **stacked**, allowing us to observe both the total value and the contribution of each category.

There are two common types of area charts:

- **Normal Area Chart** – Displays the actual values of one or more data series.
- **100% Area Chart** – Normalizes each point to 100%, emphasizing the relative contribution of each category rather than the absolute values.

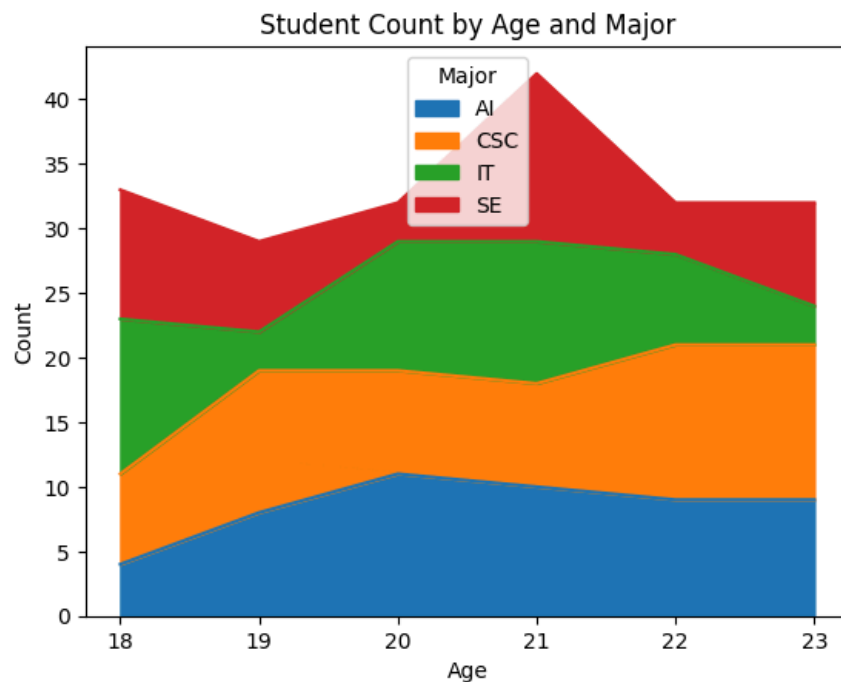
Since Seaborn does not directly support area charts, they are commonly created using Matplotlib.

Example. Normal Area Chart

```
major_age = (
    df.groupby(["Age", "Major"])
      .size()
      .unstack(fill_value=0)
)

major_age.plot.area()

plt.title("Student Count by Age and Major")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```

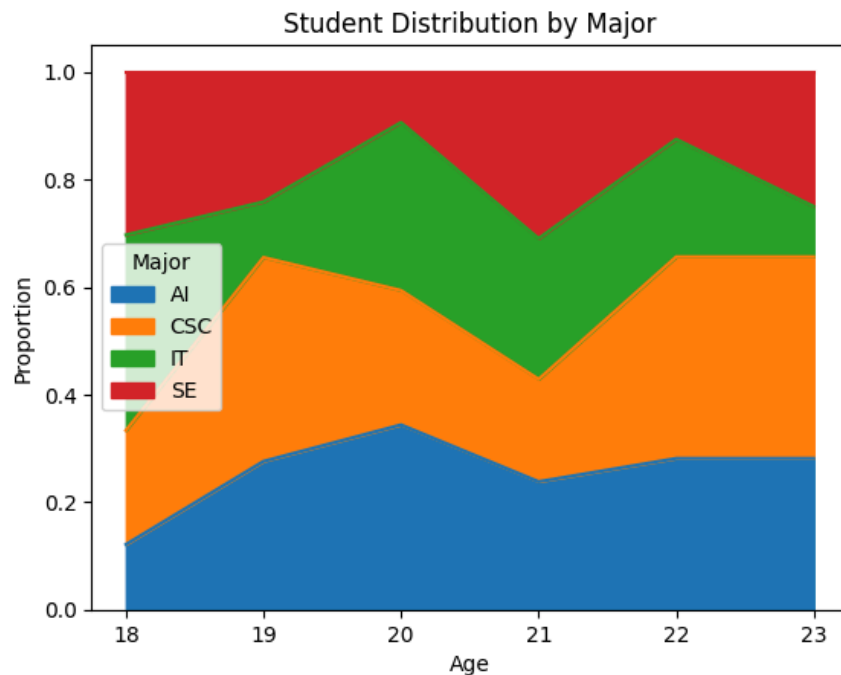


Example. 100% Area Chart

```
major_age_percent = major_age.div(
    major_age.sum(axis=1),
    axis=0
)
```

```
major_age_percent.plot.area()

plt.title("Student Distribution by Major")
plt.xlabel("Age")
plt.ylabel("Proportion")
plt.show()
```



7. Pie Chart

A **pie chart** represents how individual categories contribute to a whole. The entire circle represents **100%**, while each slice corresponds to the proportion of a category.

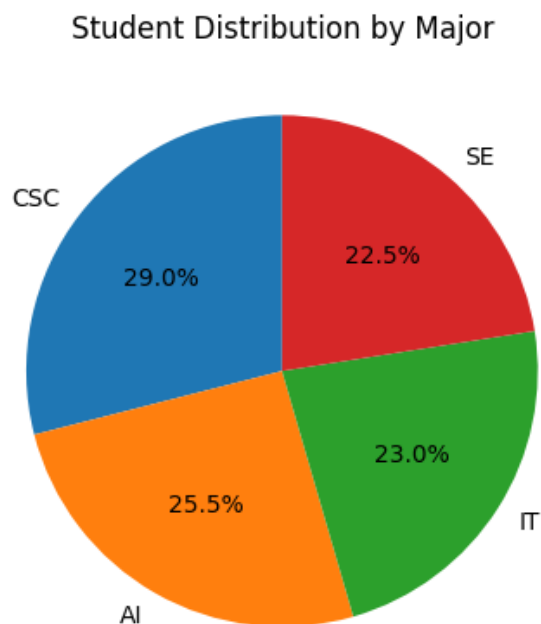
Pie charts are effective when there are only a few categories and the goal is to communicate relative proportions. However, because humans are less accurate at comparing angles than lengths, pie charts are generally **not recommended** for comparing precise values or identifying small differences between categories.

Example. Student Distribution by Major

```
major_count = df["Major"].value_counts()

plt.pie(
    major_count,
    labels=major_count.index,
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Student Distribution by Major")
plt.show()
```



Example. Donut Chart

A donut chart is a variation of the pie chart with the center removed. It conveys the same information while providing space to display additional annotations if needed.

```
major_count = df["Major"].value_counts()

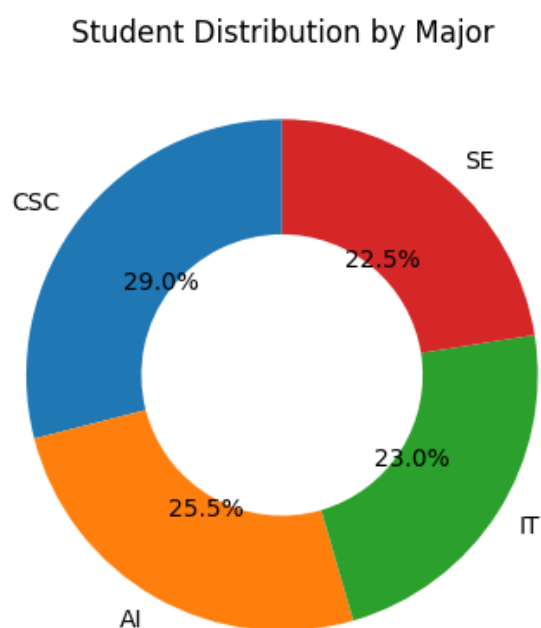
plt.pie(
```

```

major_count,
labels=major_count.index,
autopct="%1.1f%%",
startangle=90,
wedgeprops={"width":0.45}
)

plt.title("Student Distribution by Major")
plt.show()

```



Summary

This section introduced the most common **categorical plots**, which are designed to compare categories, visualize trends across groups, and display how different components contribute to a whole. Although each chart has its own strengths, selecting the appropriate visualization depends on the question being answered rather than simply choosing the most attractive figure.

In the next section, we shift our focus to **statistical plots**, including **scatter plots**, **histograms**, **box plots**, **violin plots**, **heatmaps**, and other visualizations

that help explore numerical distributions, relationships, correlations, and potential outliers within a dataset.

References

<https://seaborn.pydata.org/tutorial/categorical.html>

<https://www.geeksforgeeks.org/machine-learning/seaborn-categorical-plots/>

Next Section:

 [8.2. Data Visualization: Statistical Plots](#)